

OpenFrame GW 管理者ガイド

OpenFrame GW 7.1



Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

文書情報

文書名: OpenFrame GW 管理者ガイド

発行日: 2023年12月29日

ソフトウェアバージョン: OpenFrame GW 7.1

ガイドバージョン: v3.1.2

Website

<https://kr.tmaxsoft.com/>

<https://technet.tmaxsoft.com>

Copyright Notice

Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

Restricted Rights Legend

このソフトウェア（Tmax OpenFrame GW®）マニュアルの内容とプログラムは、日本国の著作権法および国際条約によって保護されています。マニュアルの内容とプログラムは、TmaxSoft Co., Ltd.との使用許諾契約書の下でのみ使用することができ、マニュアルは使用許諾契約で許可されている範囲を除いては、配布または複製することができません。TmaxSoftの書面による事前の承諾を得ることなく、このマニュアルの全部または一部を電子的または機械的な方法を問わず、転送、複製、配布したり、または二次的著作物を作成する等の行為を一切禁じます。

このソフトウェアのマニュアルとプログラムの使用許諾契約は、いかなる場合においても、マニュアル及びプログラムと関連する知的財産権（登録の有無を問わず）を譲渡するものと解釈されず、TmaxSoftのブランド、ロゴ、商標等の使用権限を与えるものではありません。

このマニュアルは、情報を提供する目的でのみ提供しており、これに伴う契約上の直接的ないしは間接的な責任を負わず、マニュアルの内容は法律上もしくは商業的な特定の条件が満たされることを保証しません。マニュアルの内容は、製品のアップグレード及び修正により、その内容が予告なく変更されることがあり、内容上の誤りがないことを保証しません。

Trademarks

Tmax®, Tmax OpenFrame®およびTmax OpenFrame GW®は、TmaxSoft Co., Ltd.の登録商標です。その他、記載されている会社名、製品名などは、各社の商標または登録商標です。

Open Source Software Disclaimer

この製品には、OpenSSL、RSA Data Security, Inc.、Apache FoundationおよびJean-loup Gaillyと Mark Adlerによって開発またはライセンス取得されたオープンソフトウェアが含まれています。詳細は、製品ディレクトリの\${INSTALL_PATH}/license/oss_licensesに記載されている事項を参照してください。

目次

このガイドについて	vii
第1章 紹介	1
1.1. 概要	1
1.2. 機能	1
1.3. 構造	2
1.4. 動作プロセス	4
1.5. OpenFrameとJEUSの連携	4
1.6. OpenFrame GWとOpenFrame間の文字セットの変換	5
第2章 環境設定と運用	7
2.1. ディレクトリ構造	7
2.2. OpenFrame GWの設定ファイル	8
2.2.1. ofgw.properties	9
2.2.2. vtam.properties	13
2.2.3. dhcp.properties	14
2.2.4. initScreen.txt	15
2.2.5. logback.xml	16
2.3. OpenFrame GWの運用	18
2.3.1. OpenFrame GWの起動と終了	18
2.3.2. ログ・フォーマット	21
2.4. マルチ環境の構成	23
2.4.1. OpenFrame GWのマルチノード	23
2.4.2. OpenFrame GWのマルチゲートウェイ	24
第3章 LUの自動割り当て	27
3.1. 概要	27
3.2. BEGINVTAMマクロ	27
3.2.1. LU名の設定ルール	27
3.2.2. IP設定ルール	28
3.2.3. BEGINVTAMマクロの設定ルール	29
3.3. BEGINVTAMマクロ・コマンド	29
3.3.1. DEFAULTAPPL	30
3.3.2. DEFAULTLUS	31
3.3.3. DEFAULTPRT	31
3.3.4. IPGROUP	31
3.3.5. LUGROUP	32
3.3.6. PRTGROUP	32
3.3.7. LUMAP	32
3.3.8. PRTMAP	33
3.3.9. PORT	33
付録 A. JEUSの設定ファイル	35

A.1.	概要	35
A.2.	サーバーの追加	35
A.3.	リソースの追加	36
付録 B.	Web通信	39
B.1.	概要	39
B.2.	構造	39
B.3.	動作プロセス	40
付録 C.	OFGWメッセージ・コード	43
C.1.	概要	43
C.2.	メッセージ・コード	43

図目次

[図 1.1]	OpenFrame GWの構成図	3
[図 1.2]	OpenFrame GWの動作プロセス	4
[図 1.3]	OpenFrame GWとOpenFrame、JEUSの関係	5
[図 1.4]	OpenFrame GWとOpenFrame間の文字セットの変換プロセス	5
[図 2.1]	DHCP環境でホスト名を使用してLUの割り当てを受ける方法の例	15
[図 2.2]	初期画面	16
[図 2.3]	OpenFrame GWのマルチノードの運用	23
[図 2.4]	OpenFrame GWのマルチゲートウェイの例	24
[図 B.1]	OpenFrame GW RMIサーバーの構成図	39
[図 B.2]	OpenFrame GW RMIサーバーの動作プロセス	40

このガイドについて

対象読者

本書は、Tmax OpenFrame[®](以下、OpenFrame)システムと連携してゲートウェイ機能を提供する Tmax OpenFrame GW[®](以下、OpenFrame GW)の管理者ガイドです。

同書は、OpenFrame GWを運用および管理するユーザーを対象としています。

前提知識

本書を理解するためには、OpenFrameについての基礎知識とサーバーの運用環境を含む以下のような事項についての知識が必要です。

- UNIXシステム
- WAS(Webアプリケーション・サーバー)とJEUS
- OpenFrameシステム
- メインフレームの端末または端末エミュレーター
- クライアントのWeb環境とWebブラウザー

本書の構成

本書は、計3章と付録で構成されています。

各章の主な内容は以下のとおりです。

- 第1章：紹介

OpenFrame GWの構造と特徴について説明します。

- 第2章：環境設定と運用

OpenFrame GWの環境設定と運用に必要な内容について説明します。

- 第3章: LUの自動割り当て

OpenFrame GWのLU自動割当機能を使用するために、BEGINVTAMマクロを設定する方法について説明します。

- 付録A：JEUSの設定ファイル

JEUSの設定ファイルのdomain.xmlの内容と項目について説明します。

- 付録B: Web通信

OpenFrame GW RMIサーバーの構造と特徴について説明します。

- 付録C: OFGWメッセージ・コード

OpenFrame GWから端末に出力されるメッセージ・コードについて説明します。

表記上の規則

表記	意味
<AaBbCc123>	プログラム・ソースコードのファイル名、ディレクトリ
Ctrl+C	CtrlキーとCキーを同時に押す
[Button]	GUIのボタン、メニュー名
太字	強調
「」、『』（鍵カッコ）	関連文書、あるいはガイド内の他の章および節の表示
「入力項目」	画面UI上の入力項目
ハイパーリンク	メール・アカウント、Webサイト
>	メニューの実行順
+----	サブディレクトリ/ファイル有り
----	サブディレクトリ/ファイル無し
参考	参照/注意事項
[図 1.1]	図の名前
AaBbCc123	コマンド、コマンド実行結果の画面出力、サンプル・コード
{ }	必須パラメータ値
[]	オプション・パラメータ値
	選択パラメータ値

システム要件

	要求事項
プラットフォーム	IBM AIX 5L、6L、7L
	HP-UX ia64 11.x、11i、11iV2
	Solaris 9、10、11
	Linux x86 k2.6以上、ia64 k2.6以上
ハードウェア	1GB以上のハードディスク（最低200MB）
	512MB以上のメモリを推奨
データベースのJDBC	Tibero、Oracle、MSSQLなど、JDBCを使用するデータベースのJDBCドライバー
ソフトウェア	Microsoft Windowsシステム
	Chrome、Edge（Web端末の使用時にはChromeを推奨）
	JDK 1.8以上
OpenFrame製品群	OpenFrame/Base 7.1、OpenFrame/OSC 7.1、OpenFrame/OSI 7.2、OpenFrame/AIM 7.1
WAS製品群	JEUS 7 Fix#4

参考

OpenFrame GW 7以上のバージョンは、Webアプリケーション・サーバー(WAS)上で実行される製品です。したがって、サポートするプラットフォーム情報は、OpenFrame GWが実行される WAS(JEUS 7 Fix#4)のサポート情報に従います。

関連文書

ガイド	説明
OpenFrame GW インストールガイド	OpenFrame GWのインストールについて説明しています。
OpenFrame GW Web端末ガイド	OpenFrame GWが提供するWeb端末(WebTerminal)の画面構成と使用方法について説明しています。

第1章 紹介

本章では、OpenFrame GWの構造と特徴について説明します。

1.1. 概要

OpenFrame GWは、メインフレーム・リホスト・ソリューションであるOpenFrameを構成する製品の一つです。複数の3270端末をリアルタイムで管理および監視し、オンライン製品(OSC、OSI、AIM)と端末の間でデータ処理、トランザクション、端末管理などを行います。

ユーザーは、3270プロトコルの端末とOpenFrame GWが提供するWebプロトコル端末を使用して、OpenFrameと通信することができます。

以下は、OpenFrame GWが提供するサービスです。

- アプリケーション・プログラムと端末間の接続管理
- アプリケーション・プログラムと端末間のデータ転送
- アプリケーション・プログラム間の端末共有
- 端末接続のためのLU名の自動割り当て
- ネットワーク操作の監視および管理
- Webブラウザーを介して3270エミュレーターを提供

1.2. 機能

OpenFrame GWはネットワーク通信を管理し、次のような機能をサポートしています。

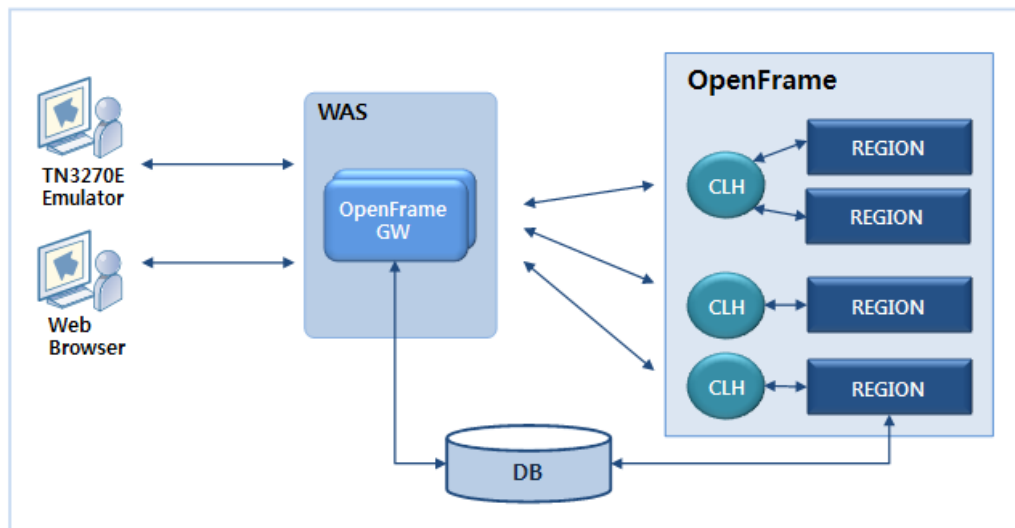
- 端末の開始と終了およびアクセス制御
 - アプリケーション・プログラムを使用して端末の接続を終了することができ、設定により端末に接続することもできます。
 - 割り当てられたリソースに基づいて端末の接続を制御します。
- 動的なリソース管理
 - 実行中にリソースの変更があった場合、データベースを介して使用可能なリソースを読み込みます。
 - 動的なリソース管理を通じて、リアルタイムで端末接続を制御します。
- データベースを介してリソースを共有
 - データベースを介して、使用可能なアプリケーション・サーバーとそれに関連するリソースを共有します。

- データベースを介して、接続中の端末とOpenFrame GWの情報を管理および共有します。
- 入出力処理
 - OpenFrame GWは、アプリケーション・プログラムと端末間のデータ転送を管理します。端末の接続はソケット通信とWeb通信を介して入出力を行い、アプリケーション・プログラムとの通信は、Tmaxを介して行います。
 - 3270プロトコルとTmaxプロトコルを使用して、OpenFrame GWの内部でそれぞれのデータを変換して入出力を行います。
- 利便性と信頼性
 - ネットワーク通信で発生するエラーの影響を低減させます。
 - OpenFrame GWにより、アプリケーション・プログラムのデータ形式を保つことができます。
- トランザクション・スケジューリング
 - アプリケーション・プログラムのトランザクションをスケジューリングします。
- Web端末の使用
 - 3270プロトコルの端末に加え、Web端末を提供します。
 - ユーザーは、Web端末を介してOpenFrame GW内での3270データ・プロトコルを維持し、3270プロトコル・エミュレーターと同様に使用することができます。
- マルチ端末とマルチサーバーの通信
 - 複数の端末との接続が可能であり、OpenFrame GWが接続を管理します。
 - 複数のアプリケーション・サーバーとの接続が可能であり、それらの接続を管理します。また、それぞれのアプリケーション・サーバーが相互に通信することができます。

1.3. 構造

OpenFrame GWは、WAS上のWebサーバーとして実装されます。データベース(Tibero)には、現在接続している端末と起動しているオンライン・リージョン(OSC、OSI、AIM、...)の情報が保存されます。

OpenFrameのオンライン・リージョンには、TmaxのCLHを介してアクセスし、データベースを介してリソースを共有します。また、Tmaxによりコネクション・スケジューリングが実行されるため、Tmaxコネクションの許容数内であれば、接続する端末の数が増えても接続することができます。



[図 1.1] OpenFrame GWの構成図

- WAS(Webアプリケーション・サーバー)

OpenFrame GWは、WAS(JEUS)上で動作します。

- DB

現在接続している端末、VTAM、およびオンライン・リージョンの情報を保存します。具体的には以下のような情報が保存されます。

- 接続しているゲートウェイ情報

現在起動しているOpenFrame GWの情報を保存します。

- 接続している端末情報

ゲートウェイごとに接続している端末の情報を保存します。この情報を基に同じLU名で接続を許可しない機能を提供し、システム管理者には現在接続している端末のリストを提供して運用上発生する問題に対応できるようにします。

- LU自動割り当てリソース情報

各ゲートウェイ(ポート)で自動で割り当てられるLUについて定義されているVTAMリソース情報を保存します。詳細については、「[第3章 LUの自動割り当て](#)」を参照してください。

- オンライン・リージョン情報

現在起動しているオンライン・リージョン(OSCOIVP1、AIM、OSI、...)の情報を保存します。

- CLH

Tmaxのクライアント・ハンドラーです。クライアントとサーバの間を中継し、サービスを提供する業務処理サーバーにサービスを要求し、サーバーへの接続および管理を行います。tpcallなどの関数によって受信されるクライアントの要求を該当するサーバーに転送します。

- REGION

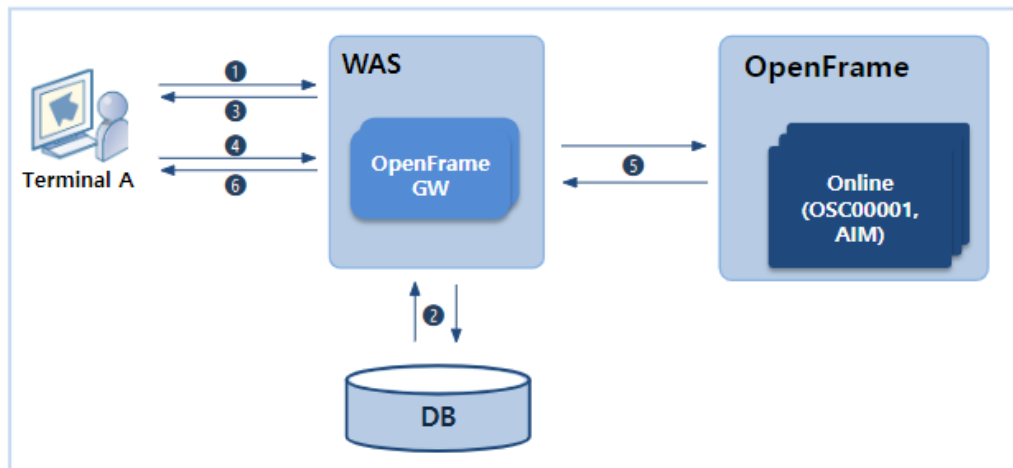
アプリケーション・プログラムを実行させ、CLHを介してOpenFrame GWと通信します。

- Webブラウザ

Webブラウザは、WAS(JEUS)の要求に対する応答に基づいてユーザーに結果を表示し、入力を受けるUIを提供します。

1.4. 動作プロセス

以下の図は、3270端末にログインして、オンライン・リージョンのトランザクションを実行するプロセスです。

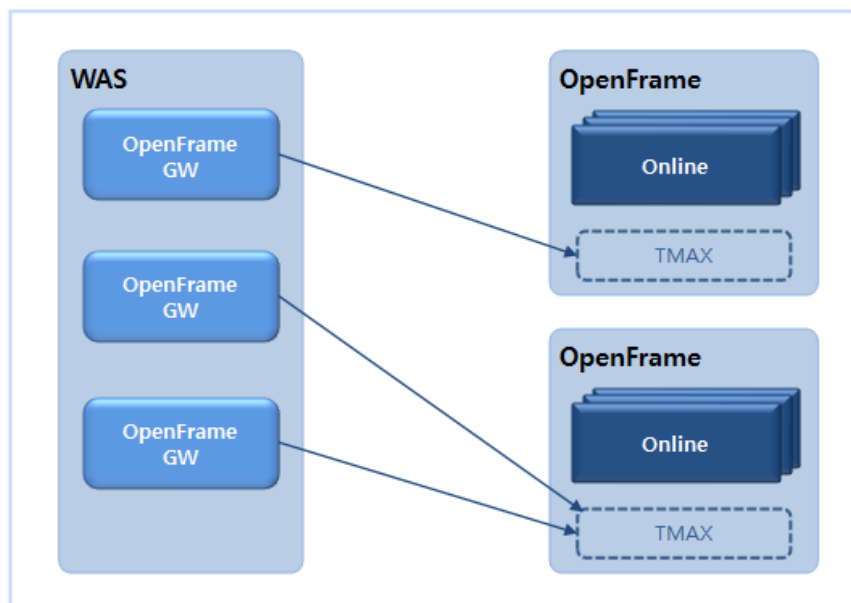


【図 1.2】 OpenFrame GWの動作プロセス

1. Terminal AがOpenFrame GWに接続した後、オンライン・リージョンへのログインを試みます（OSCおよびOSIではリージョン・サーバー名で、AIMではプロシージャ名で試行します）。
2. OpenFrame GWはDBにアクセスしてTerminal Aの整合性をチェックします。
3. データベースは、Terminal Aが有効な場合にログインを許可します。
4. ログインに成功した後、トランザクション名を入力します。
5. OpenFrame GWはオンライン・リージョンにトランザクションを渡し、実行結果を受け取ります。
6. オンライン・リージョンから受信したデータを分析して、Terminal Aに結果値を表示します。

1.5. OpenFrameとJEUSの連携

OpenFrame GWは、WAS(JEUS)上で動作します。したがって、1つのWASに1つ以上のサーバーが存在することができます。また、各サーバーにOpenFrame GWを搭載して、1つのWASに複数のOpenFrame GWを動作させることも可能です。OpenFrame GWは、同じOpenFrameに接続することも、異なるOpenFrameにそれぞれ接続することも可能です。



[図 1.3] OpenFrame GWとOpenFrame、JEUSの関係

参考

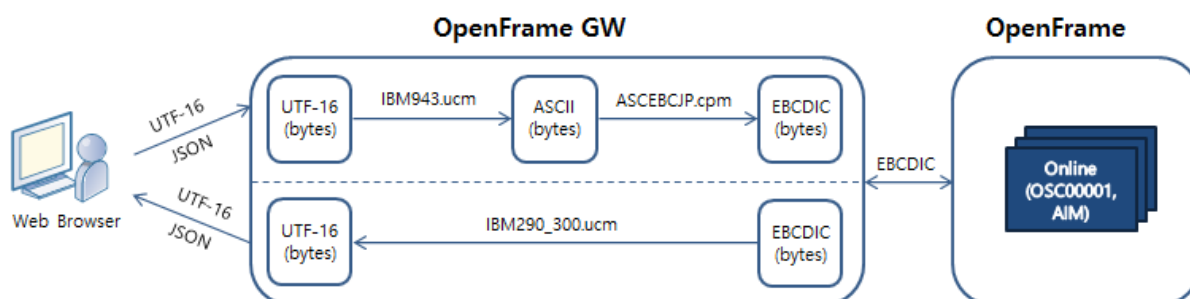
WASの詳細については、『JEUS ドメインガイド』を参照してください。

1.6. OpenFrame GWとOpenFrame間の文字セットの変換

OpenFrame GWは、3270エミュレーターだけでなく、Web端末と通信してOpenFrameで処理されたメッセージを適切な文字セット(CPM)を使用して変換してから送信する機能を提供します。

基本的に3270エミュレーターは、各国の文字をEBCDIC文字としてOpenFrame GWに送信し、Web端末はASCII文字としてデータを送信します。OpenFrameは、EBCDIC文字でのみデータを送受信できるため、Web端末の場合は、ASCII文字をEBCDIC文字に変換する必要があります。そのため、OpenFrame GWは、事前定義されたCPM文字セットを利用して処理しています。

以下は、日本語文字セット(ASCEBCJP:EBCASCJP)を利用し、Web端末がOpenFrame GWを経由してOpenFrameとデータを送受信するプロセスです。



[図 1.4] OpenFrame GWとOpenFrame間の文字セットの変換プロセス

- 送信プロセス

Web端末がUTF-16エンコードでJSONデータをOpenFrame GWに送信します。OpenFrame GWは、データをEBCDIC文字に変換するために、まず、UTF-16エンコード・データをASCII形式に変換します。そのとき、IBM943.ucm文字セットを使用してUnicodeとして表現されたデータをASCII形式に変換します。ASCIIに変換されたデータは、ASCEBCJP.cpm文字セットを使用して再びEBCDICに変換されます。

- 応答プロセス

EBCDIC形式でOpenFrame GWに送信されたデータを、再びWeb端末に送信するためにUTF-16エンコードに変換される必要があります。そのとき、IBM290_300.ucmを使用してEBCDICデータをUnicode形式に変換します。

第2章 環境設定と運用

本章では、OpenFrame GWの環境設定ファイルが格納されているディレクトリ構造と環境設定ファイル、およびOpenFrame GWの運用方法について説明します。

2.1. ディレクトリ構造

JEUS(WAS)のディレクトリ構造は以下のとおりです。

domainsフォルダーの配下にserversフォルダーが存在し、applicationフォルダー内にOpenFrame GWのバイナリが格納されます。

```
$JEUS_HOME/domains
+---- <domain_name>
+---- bin
+---- config
+---- lib
+---- servers
+---- <server_name>
+---- bin
+---- logs
+---- lib
+---- application
+---- logback.xml
+---- *.jar
+---- UninstallerData
+---- scripts
+---- webterminal
+---- ofgwconf
+---- ofgw.properties
+---- cpm
+---- license
+---- initScreen
+---- initScreen.txt
+---- vtam.properties
+---- dhcp.properties
```

servers/<server_name>/lib/application

ofgw-71-<revision>.jarとその他のライブラリが格納されています。(logback.xml設定ファイルも、このフォルダーに格納)

フォルダーまたはファイル	説明
logback.xml	ログファイルのファイル形式とログレベルを定義する設定ファイルです。詳細については、 「2.2.5. logback.xml」 を参照してください。

servers/<server_name>/lib/application/ofgwconf

OpenFrame GWの設定ファイルが格納されています。

フォルダーまたはファイル	説明
ofgw.properties	OpenFrame GWとWeb端末の環境を定義する設定ファイルです。詳細については、 「2.2.1. ofgw.properties」 を参照してください。
cpm	Web端末で使用するCPMファイルが格納されています。
license	OpenFrame GWのライセンス・ファイルが格納されています。
initScreen	端末の初期接続画面を表示するためのinitScreen.txtファイルが格納されています。詳細については、 「2.2.4. initScreen.txt」 を参照してください。
vtam.properties	マルチポートの設定と端末が各ポートに接続した場合に表示する初期画面の設定をサポートする設定ファイルです。詳細については、 「2.2.2. vtam.properties」 を参照してください。
dhcp.properties	DHCP環境でホスト名を使用してLUの自動割り当て機能をサポートする設定ファイルです。詳細については、 「2.2.3. dhcp.properties」 を参照してください。

WASに設定されているアプリケーションがデプロイされたフォルダーは、次のディレクトリに格納されます。

```
${JEUS_HOME}/domains/<domain_name>/servers/<server_name>/lib/application/  
<application_id>/
```

参考

各ディレクトリの詳細については、『JEUS アプリケーション&デプロイメントガイド』の「1.1.4. アプリケーション管理のディレクトリ構造」を参照してください。

2.2. OpenFrame GWの設定ファイル

本節では、OpenFrame GWを使用するために必要な設定ファイルの項目と設定例について説明します。

2.2.1. ofgw.properties

以下は、ofgw.propertiesファイルの各設定項目についての説明です。

```
gw.name =gateway_name
datasource.name = datasource_name
lu.port = port_no
lu.autoalloc = {yes|no}
lu.checkStatus = {yes|no}
vtamProperties = {yes|no}
dhcpProperties = {yes|no}
debug.time = {yes|no}
translate.threadPool.core = translate_threadpool_core
translate.threadPool.max = translate_threadpool_max
translate.threadPool.keepAliveTime = translate_threadpool_keepalivetime
db.threadPool.core = db_threadpool_core
db.threadPool.max = db_threadpool_max
db.threadPool.keepAliveTime = db_threadpool_keepalivetime
coroutine.threadPool.core = coroutine_threadpool_core
coroutine.threadPool.max = coroutine_threadpool_max
coroutine.threadPool.keepAliveTime = coroutine_threadpool_keepalivetime
rmi = {yes|no}
rmi.ip = rmi_deployed_ip_address
rmi.port = rmi_port_no
rmi.serviceName = rmi_service_name
rmi.proc = aim_procedure_name
rmi.smqn = aim_smqn_name
rmi.addLength = {yes|no}
rmi.logMessage = {yes|no}

cpmpath = cpmfile_path
terminal.timeout = terminal_timeout
terminal.aliveCheck = {yes|no}
terminal.aliveInterval = terminal_alive_checktime
sslEnabled = {yes|no}

tmax.retrytime = tmax_retrytime
tmax.node.list = node_name_list
tmax.node.NODE1.name = node_name
tmax.node.NODE1.ip = node_ip_address
tmax.node.NODE1.port = node_port_no
tmax.node.NODE1.min = node_min
tmax.node.NODE1.max = node_max
tmax.node.NODE1.rate = node_rate
tmax.node.NODE1.timeout = node_timeput
tmax.node.NODE1.idletime = node_idletime
```

項目	説明
gw.name	ゲートウェイ名を指定します。
datasource.name	設定ファイル(domain.xml)に指定されたデータベース名を指定します。
lu.port	3270端末が使用するポート番号を指定します。
lu.autoalloc	LUの自動割り当て機能を使用するかどうかを指定します。RMIサーバーを使用する場合は、yesに指定する必要があります。 (デフォルト値: yes)
lu.checkStatus	LUステータスのチェック機能を使用するかどうかを指定します。 (デフォルト値: no)
vtamProperties	VTAMで各ポートごとに接続可能な端末とリージョンを指定し、各初期画面を指定して使用できる機能を有効にするかどうかを指定します。(デフォルト値: no)
dhcpProperties	DHCP環境でdhcp.propertiesファイルに定義されたホスト名にマッピングされるIPでLUの自動割り当て機能を使用するかどうかを指定します。(デフォルト値: no)
debug.time	デバッグ時に、パフォーマンスをチェックするために使用する時間ログを表示するかどうかを指定します。
translate.threadPool.core	ワークスレッドの最小数を指定します。
translate.threadPool.max	ワークスレッドの最大数を指定します。
translate.threadPool.keepAliveTime	ワークスレッドの最小数より多くのスレッドが作成されている場合、アイドル状態のワークスレッドが終了されるための時間を指定します。(単位: s)
db.threadPool.core	DBスレッドの最小数を指定します。
db.threadPool.max	DBスレッドの最大数を指定します。
db.threadPool.keepAliveTime	DBスレッドの最小数より多くのスレッドが作成されている場合、アイドル状態のDBスレッドが終了されるための時間を指定します。(単位: s)
coroutine.threadPool.core	コルーチン・スレッドの最小数を指定します。
coroutine.threadPool.max	コルーチン・スレッドの最大数を指定します。
coroutine.threadPool.keepAliveTime	コルーチン・スレッドの最小数より多いスレッドが生成されている場合、アイドル状態のコルーチン・スレッドを終了させるための時間を指定します。(単位: s)
rmi	RMIサーバーを使用するかどうかを指定します。(デフォルト値: no)

項目	説明
rmi.ip	RMIサーバーのIPアドレスを指定します。（OpenFrame GWがデプロイされたサーバーのIPアドレスを指定します。）
rmi.port	RMIサーバーがデータを受信するポート番号を指定します。
rmi.serviceName	RMIサーバーがデータを受信するサーバー名を指定します。
rmi.proc	RMIサーバーがWEBAIMと通信するとき、データを送信するAIMのプロシージャを指定します。
rmi.smqn	RMIサーバーがWEBAIMと通信するとき、データを送信するAIMのSMQNを指定します。
rmi.addLength	RMIサーバーがWEBAIMと通信するとき、受信するデータの先頭に長さ情報を追加するかどうかを指定します。(デフォルト値: no)
rmi.logMessage	RMIサーバーが送受信するデータをロギングするかどうかを指定します。yesに設定する場合は、ロガー・エンコーディングをSJISに変更する必要があります。(デフォルト値: no)
cpmpath	Web端末で使用されるCPMファイルのパスを指定します。指定しない場合は、デフォルト・パスを使用します。 (デフォルト・パス: \$OFGW_HOME/ofgwconf/cpm/)
terminal.timeout	サーバーと通信していないアイドル状態のWeb端末の接続を切断するためのタイムアウトを指定します。0に指定すると、タイムアウトは使用されません。（単位: s、デフォルト値: 0）
terminal.aliveCheck	OpenFrame GWで3270エミュレーターおよびWeb端末の接続状態をチェックするために使用します。ネットワークの消失などにより接続が切断された場合に、端末リソースを自動で解放します。 – yes : terminal.aliveIntervalに設定された時間(秒)ごとに現在OpenFrame GWに接続された端末の接続状態をチェックします。 – no : OpenFrame GWに接続された端末の接続状態をチェックしません。（デフォルト値）
terminal.aliveInterval	OpenFrame GWに接続された端末の接続状態をチェックする時間間隔を指定します。（単位:s、デフォルト値: 10、最小値: 10）
sslEnabled	OpenFrame GWで3270エミュレーターと通信するとき、SSLの設定に応じて以下のように指定します。 – yes : 3270エミュレーターでSSLが有効に指定された場合は、OpenFrame GWでyesに設定して通信します。

項目	説明
	– no : 3270エミュレーターでSSLが無効に指定された場合は、OpenFrame GWでnoに設定して通信します。（デフォルト値）
tmax.retrytime	Tmaxの接続が切断されたとき、再接続を試行するまでの待機時間を指定します。（単位：ミリ秒）
tmax.node.list	Tmaxノード名を区切り文字(,)を使用して指定します。 (例：NODE1、NODE2、NODE3...)
tmax.node.[NODENAME].name	Tmaxノード名を指定します。
tmax.node.[NODENAME].ip	TmaxノードのIPを指定します。
tmax.node.[NODENAME].port	Tmaxノードのポート(\$TMAX_HOST_PORT)を指定します。
tmax.node.[NODENAME].min	Tmax接続の最小数を指定します。
tmax.node.[NODENAME].max	Tmax接続の最大数を指定します。
tmax.node.[NODENAME].rate	接続が不十分な場合、追加する数を指定します。
tmax.node.[NODENAME].timeout	接続時の待機時間を指定します。（単位: ミリ秒）
tmax.node.[NODENAME].idletime	接続数が最小数より多い場合、使用していない接続を終了する時間を指定します。（単位：秒）

参考

1. Tmaxノードの詳細については、『Tmax 管理者ガイド』を参照してください。
2. RMIサーバーの構造と特徴についての詳細は、「[付録 B. Web通信](#)」を参照してください。

以下は、ofgw.propertiesファイルの設定例です。

```

gw.name = ofgw
datasource.name = ds1
lu.port = 5556
lu.autoalloc = yes
lu.checkStatus = yes
debug.time = no
translate.threadPool.core = 5
translate.threadPool.max = 30
translate.threadPool.keepAliveTime = 60
db.threadPool.core = 5
db.threadPool.max = 30
db.threadPool.keepAliveTime = 60
coroutine.threadPool.core = 5
coroutine.threadPool.max = 30

```



```

coroutine.threadPool.keepAliveTime = 60
rmi = yes
rmi.ip = 192.144.121.11
rmi.port = 5557
rmi.serviceName = ofgwaad
rmi.proc = aadproc
rmi.smqn = aadsmqn

cpmpath =
terminal.timeout = 60
sslEnabled = no

tmax.node.list = NODE1
tmax.node.NODE1.name = NODE1
tmax.node.NODE1.ip = 192.168.37.11
tmax.node.NODE1.port = 6511
tmax.node.NODE1.min = 5
tmax.node.NODE1.max = 10
tmax.node.NODE1.rate = 2
tmax.node.NODE1.timeout = 20000
tmax.node.NODE1.idleTime = 90
tmax.retrytime = 60000

```

2.2.2. vtam.properties

vtam.propertiesファイルは、ofgw.propertiesのvtamProperties設定が「yes」の場合にのみ動作します。

参考

現在はOSCと連携する場合にのみサポートし、OSIとAIMではこの機能をサポートしません。

以下は、vtam.propertiesファイルの各設定項目についての説明です。

```

port.list = port_name
[PORT_NAME].port = port_no
[PORT_NAME].regionList = region_list
[PORT_NAME].initScreen = init_screen_filename

```

項目	説明
port.list	VTAMに設定されているポートを指定するためのポート名を設定します。 (例: PORT1,PORT2,PORT3...)
[PORT_NAME].port	PORT_NAMEのポート番号を設定します。ポート番号は、VTAMリソースに登録されているポートを使用する必要があります。

項目	説明
	複数のポート番号のいずれか1つは、ofgw.propertiesのlu.portの値と一致する必要があります。
[PORT_NAME].regionList	VTAMポートを使用して接続した端末で接続を許可するリージョン・サーバー名を設定します。（例: OSC00001, OSC00002...）
[PORT_NAME].initScreen	端末が該当のポートで接続された場合に表示する最初の画面ファイルを指定します。（\$OFGW_HOME/ofgwconf/initScreenフォルダー内に存在するファイルである必要があります）

以下は、vtam.propertiesファイルの設定例です。

```
port.list = PORT1,PORT2,PORT3
PORT1.port = 5556
PORT1.regionList = OSC0IVP1
PORT1.initScreen = initScreen.txt
PORT2.port= 5558
PORT2.regionList = OSC0IVP2
PORT2.initScreen = initScreen2.txt
PORT3.port = 5570
PORT3.regionList = OSC0IVP1,OSC0IVP2
PORT3.initScreen = initScreen3.txt
```

2.2.3. dhcp.properties

dhcp.propertiesファイルは、ofgw.propertiesのdhcpProperties設定が「yes」の場合にのみ動作します。

以下は、dhcp.propertiesファイルの各設定項目についての説明です。

```
hostnames = [HOST_NAME1]:[IP1],[HOST_NAME1]:[IP2],[HOST_NAME2]:[IP3],...,\
            [HOST_NAME5]:[IP10],...
```

項目	説明
hostnames	DHCP環境でWeb端末を介して入力されたホスト名にマッピングされるIP値を保存します。 – [HOST_NAME]: Web端末を介して入力されるホスト名を指定します。 – [IP]: ホスト名にマッピングされるIP値を指定します。

以下は、dhcp.propertiesファイルの設定例です。

```
hostnames=hostname1:192.168.11.23,hostname2:192.168.14.32,hostname2:192.168.14.33,\
hostname1:192.168.11.31,hostname3:192.168.17.87
```

DHCPでのLU割り当て機能の使用方法

DHCP環境ではIPが動的に変わるため、固定IPを使用してLUを割り当てるには、別の方法を使用する必要があります。OpenFrame GWでは、Web端末のクエリ・パラメータとして入力されるホスト名を使用して固定IPを抽出し、このIPを使用してLUを割り当てる方法をサポートしています。

以下は、dhcpPropertiesを「yes」に設定した後、Web端末を使用してDHCP環境でLUの割り当てを受ける方法の例を示しています。

```
127.0.0.1:8088/webterminal/?host=YUc5emRHNWhiV1V4
```

[図 2.1] DHCP環境でホスト名を使用してLUの割り当てを受ける方法の例

hostをキーとして、LU割り当てに使用するホスト名をBase64でエンコード(hostname1 → aG9zdG5hbWUx → YUc5emRHNWhiV1V4)を2回行い、値として設定してからWeb端末に接続します。

その後、従来の方で接続を試みると、OpenFrame GWではdhcpPropertiesがyesに設定されていることを確認し、入力されたホスト名(hostname1)を使用してdhcp.propertiesからIPを取得します。そして、接続したWeb端末のIP(127.0.0.1)は無視し、取得したIP(192.168.11.23、192.168.11.31)を使用してLU割り当てを行います。

参考

ホスト名を使用したLU割り当て方法は、上記の方法でWeb端末で接続した場合にのみ有効であり、RMIまたは3270エミュレーターで接続した場合は、従来のLU割り当て方法を使用します。

2.2.4. initScreen.txt

initScreen.txtファイルは、エディターを用いて端末に初めて接続するときに表示される画面構成を保存します。ファイルに作成された内容が端末画面に表示され、内容の最後の次に入力フィールドとカーソルが位置します。ファイルに以下のように設定すると、端末に同じ間隔と空白が表示されます。

ofgw.propertiesのvtamProperties項目を「yes」に設定してVTAMポート別に接続できるようにした場合は、vtam.propertiesの[PORT_NAME].initScreenに設定されたファイルの画面が表示されます。vtamProperties項目を「no」に設定した場合は、initScreen.txtの画面が表示されます。

```
@ THIS TERMINAL CONNECTED TO THE OPENFRAME GW
```



```

<appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
  <encoder>
    <charset>UTF-8</charset>

<pattern>[%d{yyyy-MM-dd}T%d{HH:mm:ss.SSSSS}][OFGW|%-24logger{0}][%-24t{1}][%. -1level]
%msg%n</pattern>
  </encoder>
</appender>
<property name="USER_HOME" value="/home/user/logs" />
<appender name="ROLLING" class="ch.qos.logback.core.rolling.RollingFileAppender">

  <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
    <fileNamePattern>${USER_HOME}/ofgw-%d{yyyy-MM-dd}.%i.log</fileNamePattern>

    <timeBasedFileNamingAndTriggeringPolicy
class="ch.qos.logback.core.rolling.SizeAndTimeBasedFNATP">
      <maxFileSize>300MB</maxFileSize>
    </timeBasedFileNamingAndTriggeringPolicy>
    <maxHistory>30</maxHistory>
  </rollingPolicy>
  <encoder>

<pattern>[%d{yyyy-MM-dd}T%d{HH:mm:ss.SSSSS}][OFGW|%-24logger{0}][%-24t{1}][%. -1level]
%msg%n</pattern>
  </encoder>
</appender>
<logger name="com.tmax.ofgw.region.LUManager" level="DEBUG" />
<logger name="com.tmax.ofgw.Main" level="INFO" />
<logger name="com.tmax.ofgw.region.Region" level="ERROR" />
<logger name="com.tmax.webtnio" level="OFF" />
<root level="DEBUG">
  <appender-ref ref="STDOUT" />
  <appender-ref ref="ROLLING" />
</root>
</configuration>

```

以下は、プロパティ・ファイルの項目についての説明です。

項目	説明
<configuration>	scanとscanperiod項目を設定すると、動的にログレベルを変更することができます。
<appender>	LogBackで提供する形式でロガーのフォーマットを追加することができます。（イベントごとにログを記録することができます。） – ConsoleAppender：コンソールで表示される形式を提供します。

項目	説明
	– RollingFileAppender：ファイルの最大容量を設定し、索引でログファイルを作成します。
<charset>	ログファイルのエンコード形式を設定します。ofgw.propertiesのrmi.addLengthがyesに設定された場合は、その属性をSJISに設定する必要があります。(デフォルト値: UTF-8)
<property>	プロパティに設定した値は、変数のように使用することができます。このファイルでのUSER_HOMEは、ログファイルのパスとして使用されます。
<fileNamePattern>	ログファイル名のパターンを指定します。 デフォルト値は、日付形式と索引("%i")が追加されます。
<maxFileSize>	1ファイルあたりの最大容量を指定します。
<maxHistory>	ファイルのログを記録する最大日数を指定します。この値を超えると、自動的に最も古いログから順に削除されます。
<pattern>	表示されるログの形式を指定します。
<logger name>	<root>に設定されたログレベルとは別に、特定のクラスを指定してログレベルを指定します。
<root>	ログレベルを指定します。<appender>を追加して、該当のロガー・フォーマットのログレベルとして適用されます。 <logger name>が記述されていない場合、デフォルトで<root>に記述されたログレベルで各クラスのログレベルが設定されます。<logger name>が設定されたクラスの場合は、<root>に記述されたログレベルを上書きします。

参考

上記の項目はデフォルトで提供される項目です。したがって、管理者が環境に合わせてカスタマイズして使用する必要があります。追加項目については、LogBack関連のガイドを参照してください。

2.3. OpenFrame GWの運用

本節では、OpenFrame GWの起動と終了およびログ・フォーマットについて説明します。

2.3.1. OpenFrame GWの起動と終了

OpenFrame GWを起動および終了する手順について説明します。

起動

JEUSのDAS上にOpenFrame GWをインストールした場合、DASを起動すると<lifecycle-invocation>属性によってOpenFrame GWも一緒に起動されます。DASを起動するスクリプトはstartDomainAdminServerであり、DASのドメイン名やJEUSを開始する権限を持つユーザー名とパスワードが必要です。

JEUS_HOME/binの下で実行する場合はこれらの値がすべて必要ですが、DASのDOMAIN_HOMEの下で実行する場合は、ドメイン名を省略することができます。

以下は、場所に応じたOpenFrame GWの起動です。

1. startDomainAdminServerでサーバーを起動します。

```
$JEUS_HOME/bin/startDomainAdminServer -domain <domain_name> -u <user_name> -p <password>
```

または

```
$JEUS_HOME/domains/<domain_name>/bin/startDomainAdminServer -u <user_name> -p <password>
```

2. startManagedServerでサーバーを起動します。

```
$JEUS_HOME/domains/<domain_name>/servers/<server_name>/bin/startManagedServer -u <user_name> -p <password>
```

または

```
$startManagedServer -domain <domain_name> -server <server_name> -u <user_name> -p <password> -dasurl <das_ip:das_baseport>
```

3. jpsツールを使用して、DomainAdminServerBootstrapperとServerBootstrapperが正常に起動されたことを確認します。

```
$jps
23541 Jps
22797 DomainAdminServerBootstrapper
22981 ServerBootstrapper
```

終了

OpenFrame GWを終了するには、OpenFrame GWがインストールされているサーバーを終了する必要があります。

1. stopServerコマンドを使用してManaged Serverサーバーを終了します。

```
$JEUS_HOME/bin/stopServer -u <user_name> -p <password> -host <server_ip:server_baseport>
```

2. stopServerコマンドを使用してDomainAdminServerサーバーを終了します。

```
$JEUS_HOME/bin/stopServer -u <user_name> -p <password> -host  
<server_ip:server_baseport>
```

参考

WebAdmin、jeusadmin、またはコマンドを使用してJEUSを起動および終了することができます。本書では、UNIX環境での起動と終了についてのみ説明しています。他の方法についての詳細は、『JEUS ドメインガイド』または『JEUS WebAdminガイド』を参照してください。

起動と終了時のデータベース操作

OpenFrame GWは、OFM_BASE_VTAM_GATEWAY、OFM_BASE_VTAM_ACTIVE_LU表に情報を更新します。

起動時には、以前の異常終了により、情報が残っているかどうかを確認します。残っている場合はその情報を削除してから、新たに起動されるOpenFrame GW情報を更新します。一方、終了時には、ゲートウェイに接続されている端末の接続が切断されると同時に、そのサーバーに登録されているOpenFrame GWの情報と端末の情報をデータベースから削除します。

以下は、**vtamadm**ツールを使用してOpenFrame GWの情報を確認する方法です。

● ゲートウェイ情報の確認

以下は、OpenFrame GWの情報を確認するコマンドの使用方法です。

```
$ vtamadm -w  
  
VTAM Gateway List  
  
=====
```

NO.	GATEWAY_NAME	CLID	IP_ADDR:PORT	NODENAME
1	ofgw_A	3	192.168.105.208:5668	NODE1

```
=====
```

● 端末LU情報の確認

以下は、OpenFrame GWに接続されている端末のLU情報を確認するコマンドの使用方法です。

```
$ vtamadm -l  
  
Active LU information connected to VTAM  
  
=====
```

NO.	LU	TYPE	CLID	NODENAME	IP_ADDR
	LOGON_REGION	USERID			

```
=====
```


1	OSC00001	OSC REGION	NODE1	
2	OIVPTRM1	IBM-3278-2-E 5	NODE1	192.168.105.208
	OSC00001	ROOT		
3	TSO	TSO	NODE1	

参考

vtamadmコマンドは、OpenFrameが提供するツールです。詳細については、『OpenFrame ツールリファレンスガイド』を参照してください。

オンライン・リージョンの起動と終了

基本的にOpenFrame GWは、OpenFrameのリージョン・サーバーの起動と終了に関係なく、起動状態を維持します。OpenFrame GWの実行中にオンライン・リージョンが起動されると、データベースから該当するリージョン情報を取得します。

終了時には、オンライン・リージョンに接続している(ログオンされたLU)端末の接続を切断し、データベースからも関連情報を削除します。

OpenFrameエンジンがすべて終了すると、Tmax接続が失われ、以下のようなログが発生します。

```
[2022-06-22T14:49:07.00976][OFGW|WebtChannelHandlerImpl ][Main [server1-67]
][I] Close connection for node[NODE1], clid=[246]
[2022-06-22T14:49:07.00982][OFGW|WebTerminal ][Main [server1-67]
][D] WebTerminal disconnect status=transaction_status_none
[2022-06-22T14:49:07.00984][OFGW|WebSocketEndPoint ][Main [server1-67]
][D] handleClose(...)
[2022-06-22T14:49:07.00985][OFGW|WebtChannelHandlerImpl ][Main [server1-67]
][I] System channel[NODE1] (clid=[246])
```

このとき、ofgw.properties設定ファイルのtmax.retrytimeに指定された時間が過ぎてから再接続を試みます。そのとき、OpenFrameが起動している必要があります。

2.3.2. ログ・フォーマット

以下は、logback.xmlにデフォルトで提供されるログ・フォーマットです。使用環境に合わせて変更して使用します。

```
[時間] [ログレベル] [スレッド] [ロガー] [メッセージ]
```

項目	説明
[時間]	「時間:分:秒.ミリ秒」形式で表示されます。
[ログレベル]	指定されたログレベルを表示します。OpenFrame GWでは、基本的にERROR、WARN、INFO、DEBUGの4つのレベルのログを表示します。

項目	説明
	次は、各ログレベルについての説明です。 – ERROR: モジュールが異常終了したか、エラーが発生した場合です。 – WARN: モジュールの動作に問題があるが、GWの動作には影響を与えない場合です。 – INFO: モジュールの動作の成功または失敗に関係なく、実行されたタスクの重要情報やタスクの開始と終了を通知する情報を表示します。 – DEBUG: 各モジュールの動作について、INFOレベルより詳しい情報を表示します。 ログレベルの順は、ERROR < WARN < INFO < DEBUGです。上位レベルのログは、下位レベルのログを含みます。たとえば、ログレベルがINFOに設定されている場合は、ERRORとWARNレベルのログまで表示されます。
[スレッド]	現行のスレッド名です。ロギングするプロセス(サーバーまたはランチャー)とスレッド番号をハイフン(-)で区切って指定します。
[ロガー]	「ロガー名+{length}」です。lengthは最大桁数です。
[メッセージ]	ログメッセージのエイリアスです。

以下は、logback.xml設定ファイルにログの表示パターンを定義した例です。

```
<pattern>[%d{yyyy-MM-dd}T%d{HH:mm:ss.SSSSS}][OFGW|%-24logger{0}][%-24t{1}][%. -1level]
%msg%n</pattern>
```

```
[2022-06-22T14:49:06.00599][OFGW|WebtChannelHandlerImpl ][Main [server1-67]
][D] Node name=NODE1
[2022-06-22T14:49:06.00599][OFGW|WebtChannelHandlerImpl ][Main [server1-67]
][D] Msg for OFGW
[2022-06-22T14:49:06.00600][OFGW|WebtChannelHandlerImpl ][Main [server1-67]
][D] Tmax msg type: tpsendto
[2022-06-22T14:49:06.00600][OFGW|VtamMessage ][Main [server1-67]
][D] [VtamMessage] vtam header: msgtype(aim(104)) clid(0) gwclid(0) regionclid(0)
[2022-06-22T14:49:06.00600][OFGW|VtamMessage ][Main [server1-67]
][D] [VtamMessage] msgflag : 0, ipaddr :
...
[2022-06-22T14:49:06.00650][OFGW|TranslateWorker ][Worker-3 [server1-79]
][D] Send to TransWorker channel there is something to work for terminal(id=13)
[2022-06-22T14:49:06.00651][OFGW|TransWorkerChImpl ][Worker-3 [server1-79]
][D] WorkerThread > TransWorker channel
[2022-06-22T14:49:06.00651][OFGW|TransWorkerChImpl ][Worker-3 [server1-79]
][D] writeQ for Web count=1
[2022-06-22T14:49:06.00654][OFGW|TransWorkerChImpl ][Worker-3 [server1-79]
```

```

    ][I] < Send to WebTerminal(id=13, len=2468)
[2022-06-22T14:49:06.00654][OFGW|TransWorkerChImpl          ][Worker-3 [server1-79]
    ][D] < Send data(id=13):
...
[2022-06-22T14:49:06.00655][OFGW|TranslateWorker          ][Worker-3 [server1-79]
    ][D] translation(id=13) done, WorkerThread > TransWorkChannel done

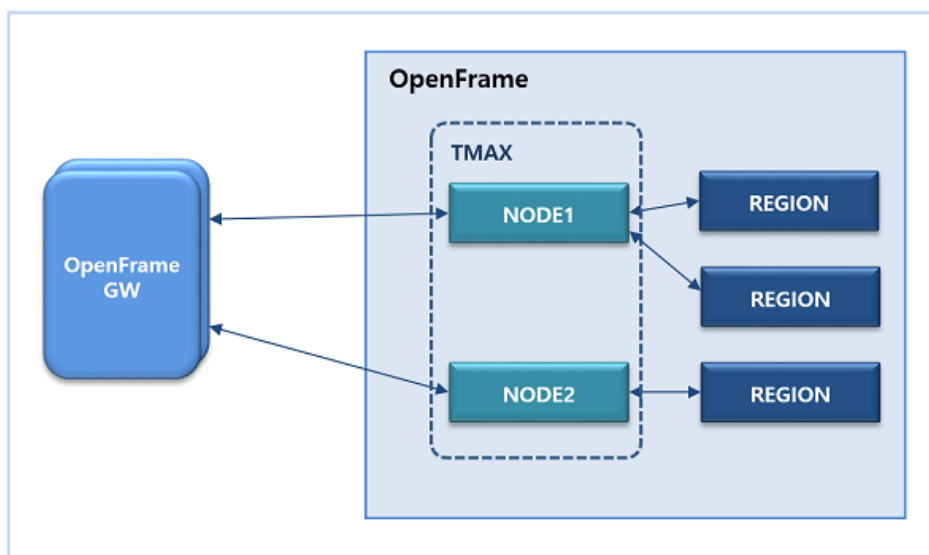
```

2.4. マルチ環境の構成

本節では、マルチ環境を構成する方法について説明します。

2.4.1. OpenFrame GWのマルチノード

負荷分散環境を構築するために、OpenFrame GWはOpenFrameにマルチノードを構成して運用することができます。以下は、その構成図です。



[図 2.3] OpenFrame GWのマルチノードの運用

マルチノードを構成する場合、ofgw.propertiesファイルに以下のとおり設定します。ファイルの詳細については、「[2.2.1. ofgw.properties](#)」を参照してください。ただし、1つのOpenFrame GWから異なるOpenFrameにインストールされたノード環境への接続はサポートしていません。

```

tmax.node.list = NODE1, NODE2
tmax.node.NODE1.name = NODE1
tmax.node.NODE1.ip = 192.168.33.1
tmax.node.NODE1.port = 6300
tmax.node.NODE1.min = 2
tmax.node.NODE1.max = 6
tmax.node.NODE1.rate = 3
tmax.node.NODE1.timeout = 10000

```

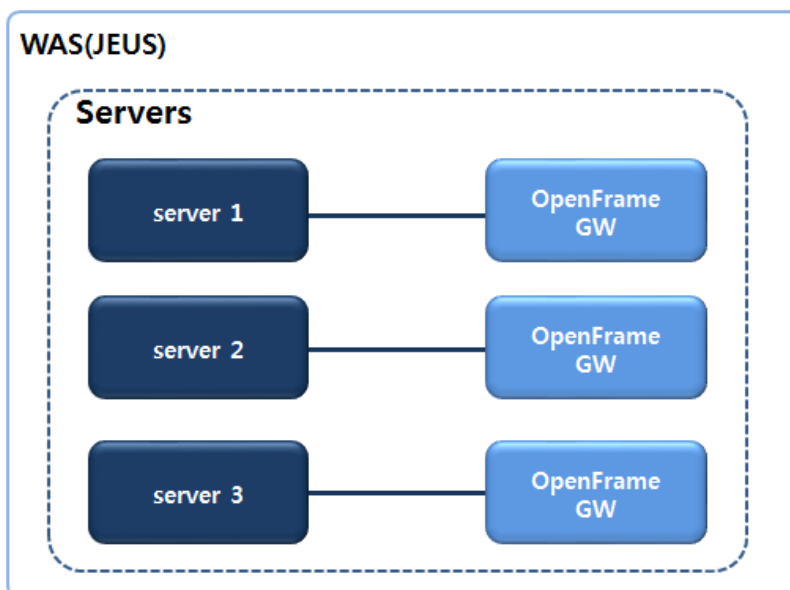
```
tmax.node.NODE1.idletime = 30000

tmax.node.NODE2.name = NODE2
tmax.node.NODE2.ip = 192.168.33.1
tmax.node.NODE2.port = 6400
tmax.node.NODE2.min = 2
tmax.node.NODE2.max = 6
tmax.node.NODE2.rate = 3
tmax.node.NODE2.timeout = 10000
tmax.node.NODE2.idletime = 30000 ...
```

Tmaxは、OpenFrame GWとオンライン・リージョン間の通信および接続スケジューリングを担当し、OpenFrame GWは、業務スケジューリングを担当します。OpenFrame GWは、データベースに保存されている各オンライン・リージョンごとのマッピング情報を読み込んでスケジューリングします。

2.4.2. OpenFrame GWのマルチゲートウェイ

WASに複数のOpenFrame GWを実行する場合、以下の図のようにサーバーごとにOpenFrame GWを1つずつ実行することができます。



【図 2.4】 OpenFrame GWのマルチゲートウェイの例

上記のような環境を構成するために、JEUSサーバーの設定は、**domain.xml**(\$JEUS_HOME)/domains/<domain_name>/config)設定ファイルに以下のように記述します。domain.xmlの各項目についての詳細は、「[付録 A. JEUSの設定ファイル](#)」を参照してください。OpenFrame GWのバイナリは、追加したJEUSサーバーのlib/applicationディレクトリに格納する必要があります。

```
<servers>
  <server>
```

```

<name>server1</name>
<lifecycle-invocation>
  <class-name>com.tmax.ofgw.Main</class-name>
  <invocation>
    <invocation-method>
      <method-name>init</method-name>
    </invocation-method>
    <invocation-type>READY</invocation-type>
  </invocation>
</lifecycle-invocation>
<lifecycle-invocation>
  <class-name>com.tmax.ofgw.Main</class-name>
  <invocation>
    <invocation-method>
      <method-name>shutdown</method-name>
    </invocation-method>
    <invocation-type>BEFORE_UNDEPLOY</invocation-type>
  </invocation>
</lifecycle-invocation>
</server>
<server>
  <name>server2</name>
  <lifecycle-invocation>
    <class-name>com.tmax.ofgw.Main</class-name>
    <invocation>
      <invocation-method>
        <method-name>init</method-name>
      </invocation-method>
      <invocation-type>READY</invocation-type>
    </invocation>
  </lifecycle-invocation>
  <lifecycle-invocation>
    <class-name>com.tmax.ofgw.Main</class-name>
    <invocation>
      <invocation-method>
        <method-name>shutdown</method-name>
      </invocation-method>
      <invocation-type>BEFORE_UNDEPLOY</invocation-type>
    </invocation>
  </lifecycle-invocation>
</server>
...省略
</servers>

```

参考

サーバーの追加についての詳細は、『JEUSサーバーガイド』の「2.2.サーバーの追加」を参照してください。

第3章 LUの自動割り当て

本章では、OpenFrame GWのLU自動割り当て機能を使用するためにBEGINVTAMマクロを設定する方法について説明します。

3.1. 概要

LUの自動割り当ては、接続する端末のIPを基準に、データセットに保存されている情報を基に端末にLU名を割り当てたり、端末のLU名がすでに設定されている場合には、整合性をチェックして接続の許可を決定したりする機能です。この機能を使用するには、BEGINVTAMマクロ文を使用してIP-LUマッピング情報を構成する必要があります。

以下は、VTAM自動割り当ての設定例です。

```
BEGINVTAM
PORT 15400
DEFAULTLUS DFLT0000..DFLT9999..FFFFNNNN ENDDEFAULTLUS
LUGROUP LUGRP1 UGSS0000..UGSS9999..FFFFNNNN ENDLUGROUP
IPGROUP IPGRP2 1.1.1.1..255.255.255.255 ENDIPGROUP
LUMAP LUGRP1 IPGRP2
ENDVTAM
```

3.2. BEGINVTAMマクロ

BEGINVTAMマクロ文でIPグループとLUグループをマッピングして、端末にLU名を自動で割り当てることができます。また、BEGINVTAMマクロ文のコマンド文を組み合わせ、デフォルト・リージョンの自動ログオン機能を使用することもできます。

参考

現在デフォルト・リージョンの自動ログオン機能は、OSCでのみ提供しています。

デフォルト・リージョンは、ユーザーがリージョンを指定しなかった場合に、端末が接続時に自動でログオンするリージョンです。端末がLU名を指定して接続を試みる場合には、整合性チェック機能も提供します。

マクロを作成するには、以下に説明するBEGINVTAMマクロの設定ルールを遵守してください。

3.2.1. LU名の設定ルール

BEGINVTAMマクロ文で各種コマンドのオプションとしてLU名を指定する場合は、以下のルールを参照してください。

- 指定するLU名の最初の文字は、必ずA-Z、@、#、または\$にします。残りの桁は、これらの文字に加えて、数字0-9を使用できます。出力できない文字は使用を控えてください。
- LU名の範囲を設定することもできます。

範囲の設定形式は、以下のとおりです。

```
LowerRange..UpperRange[..rangerule]
```

- 空白文字は使用できません。
- LowerRange、UpperRange、rangeruleの長さ（文字列数）は一致する必要があり、8文字を超えることはできません。
- UpperRangeはLowerRangeより大きい必要があります。（文字のASCII値を比較）
- rangeruleは、以下のようなワイルドカードの使用ルールに従います。

項目	説明
F	固定文字であり、変更されません。
A	英大文字のみ指定できます。
N	数字のみ指定できます。
B	英大文字と数字のみ指定できます。
X	16進数の数字と英大文字のみ指定できます。(0~F)
?	@、#、\$を含む数字と英大文字のみ指定できます。

以下の例は、TCPは固定文字のFFFであり、Nは数字0-9を、Xは16進数(0~F)を、?は0-9、A-Z、@、#、\$を、AはA-Z、Bは0-9、A-Zをそれぞれ指定できることを示しています。

```
TCP000A0..TCP9F$ZZ..FFFNX?AB
```

以下は、AB100~AB999(900)、BB000~BB999(1000)、およびCB000~CB299(300)の合計2200のLU名を指定する例です。

```
AB100..CB299..AFNNN
```

3.2.2. IP設定ルール

以下は、IPアドレスを表記する方法です。IPv4アドレスのみ有効です。

```
XXX.XXX.XXX.XXX
```

以下は、合計255のIPアドレスを設定する例です。

```
1.1.1.1..1.1.1.255
```


3.2.3. BEGINVTAMマクロの設定ルール

BEGINVTAMマクロ文は以下のようなルールに従って使用します。

- BEGINVTAM文内(~ENDVTAM)で1つ以上のポートを指定したい場合には、必ずBEGINVTAM文の最初のコマンドに指定する必要があります。
- 自動ログインするためには、LUGROUPに指定するLU名またはLU範囲内のLU名が、接続するリージョンに登録されている必要があります。
- グループ名を指定する際には、以下の文字を除外します。

ASCII値	HEX値	説明
.	2E	ピリオド
*	2A	アスタリスク
;	3B	セミコロン
,	2C	コンマ
=	3D	等号

- LUMAPに使用されるLUグループ名とIPグループ名は必ず事前に定義されている必要があります。
- LUGROUPは使用されるIPGROUPとマッピングされる必要があります。
- 最初の列でセミコロン(;)で始めると、コメントとして処理されます。
- ゲートウェイが当該情報(BEGINVTAM文)を起動中に取得するためには、**vtamgen**ツールを使用してあらかじめvtam.confの[VTAM]セクションのDSNAMEに設定されたデータセットに情報を保存する必要があります。vtamgenツールについての詳細は、『OpenFrame ツールリファレンスガイド』を参照してください。
- 異なるIPグループで重複してIPを定義した場合は、先に定義されたものが適用され、以降のものは警告メッセージが返されます。

たとえば、以下のように設定した場合、2番目のIPグループXYZに定義された「2.2.2.2」は無視されます。

```
IPGROUP ABC 1.1.1.1 2.2.2.2 ENDIPGROUP
IPGROUP XYZ 2.2.2.2 3.3.3.3 ENDIPGROUP
```

3.3. BEGINVTAMマクロ・コマンド

以下は、BEGINVTAMマクロ文で使用されるコマンドの説明です。

コマンド	説明
DEFAULTAPPL	端末とゲートウェイが接続する際に、接続を試みるリージョンを指定します。

コマンド	説明
DEFAULTLUS	LUグループを設定していない場合に使用されるデフォルトのLUグループを指定します。
DEFAULTPRT	プリンターLUグループを設定していない場合に使用されるデフォルトのプリンターLUグループを指定します。（解析のみサポートしています。）
IPGROUP	IPグループを指定します。
LUGROUP	LUグループを指定します。
PRTGROUP	プリンターLUグループを指定します。（解析のみサポートしています。）
LUMAP	IPグループとLUグループをマッピングします。
PRTMAP	IPグループとプリンターLUグループのLUをマッピングします。（解析のみサポートしています。）
PORT	ゲートウェイのポート番号を指定します。

参考

上記以外のALLOWAPPL、DEFAULTLUSSPEC、DEFAULTPRTSPEC、DESTIPGROUP、DESTIPGROUP、HNGROUP、INTERTCP、LINEMODEAPPL、LINKGROUP、MONITORGROUP、MONITORMAP、PARMSGROUP、PARMSMAP、PRTDEFAULTAPPL、RESTRICTAPPL、USERGROUP、USSTCPコマンドはサポートしません。

3.3.1. DEFAULTAPPL

LUMAPのDEFAPPLにリージョンが設定されていない場合に使用されるデフォルト・リージョン名を指定します。

参考

現在DEFAULTAPPL機能は、OSCでのみ提供しています。

● 使用方法

```
DEFAULTAPPL application_name
```

項目	説明
<i>application_name</i>	設定するリージョンの名前を指定します。

3.3.2. DEFAULTLUS

LUグループを設定していない場合に使用されるデフォルトのLUグループを指定します。

- 使用方法

```
DEFAULTLUS luname|luname1..luname2[..range_rule] ENDDEFAULTLUS
```

項目	説明
<i>luname</i>	設定するLU名を指定します。
<i>luname1</i> .. <i>luname2</i>	LU名の範囲を指定します。
<i>range_rule</i>	LU名のルールを指定します。

3.3.3. DEFAULTPRT

プリンターLUグループを設定していない場合に使用されるデフォルトのプリンターLUグループを指定します。

- 使用方法

```
DEFAULTPRT prt_name|prt_name1..prt_name2[..range_rule] ENDDEFAULTPRT
```

項目	説明
<i>prt_name</i>	設定するプリンター名を指定します。
<i>prt_name1</i> .. <i>prt_name2</i>	プリンター名の範囲を指定します。
<i>range_rule</i>	プリンター名のルールを指定します。

3.3.4. IPGROUP

IPグループを指定します。

- 使用方法

```
IPGROUP ip_group_name ip_addr|ip_range1..ip_range2 ENDIPGROUP
```

項目	説明
<i>ip_group_name</i>	LUMAPで使用するIPグループ名を指定します。
<i>ip_addr</i>	IPv4アドレスを指定します。
<i>ip_range1</i> .. <i>ip_range2</i>	IPv4アドレスの範囲を指定します。

3.3.5. LUGROUP

LUグループを指定します。

- 使用方法

```
LUGROUP lu_group_name lu_name|lu_name1..lu_name2[..range_rule] ENDLUGROUP
```

項目	説明
<i>lu_group_name</i>	LUMAPで使用するLUグループ名を指定します。
<i>lu_name</i>	LU名を指定します。
<i>lu_name1</i> .. <i>lu_name2</i>	LU名の範囲を指定します。
<i>range_rule</i>	LU名のルールを指定します。

3.3.6. PRTGROUP

プリンターLUグループを指定します。

- 使用方法

```
PRTGROUP lu_group_name lu_name|lu_name1..lu_name2[..range_rule] ENDPRTGROUP
```

項目	説明
<i>lu_group_name</i>	PRTMAPで使用するLUグループ名を指定します。
<i>lu_name</i>	LU名を指定します。
<i>lu_name1</i> .. <i>lu_name2</i>	LU名の範囲を指定します。
<i>range_rule</i>	LU名のルールを指定します。

3.3.7. LUMAP

IPグループとLUグループをマッピングします。

- 使用方法

```
LUMAP luname|lu_group_name ip_addr|ip_group_name [DEFAPPL application_name]  
[GENERIC | SPECIFIC]
```

項目	説明
<i>luname</i>	LU名を指定します。
<i>lu_group_name</i>	LUグループ名を指定します。

項目	説明
<i>ip_addr</i>	IPv4アドレスを指定します。
<i>ip_group_name</i>	IPグループ名を指定します。
<i>application_name</i>	リージョン名を指定します。（OSCでのみ提供）
GENERIC	デフォルトでGENERICオプションを適用します。（端末にLUを割り当てる）
SPECIFIC	端末がLU名を指定した場合、そのLUMAPで検索します。

3.3.8. PRTMAP

IPグループとプリンターPRTGROUPのLUグループをマッピングします。

- 使用方法

```
PRTMAP luname | lu_group_name ip_addr | ip_group_name [DEFAPPL application_name]
[GENERIC | SPECIFIC]
```

項目	説明
<i>luname</i>	LU名を指定します。
<i>lu_group_name</i>	LUグループ名を指定します。
<i>ip_addr</i>	IPv4アドレスを指定します。
<i>ip_group_name</i>	IPグループ名を指定します。
<i>application_name</i>	リージョン名を指定します。（OSCでのみ提供）
GENERIC	デフォルトでGENERICオプションを適用します。（端末にLUを割り当てる）
SPECIFIC	端末がLU名を指定した場合、そのLUMAPで検索します。

3.3.9. PORT

ゲートウェイのポート番号を指定します。

- 使用方法

```
PORT num [, num]
```

項目	説明
<i>num</i>	ポート番号を指定します。

付録 A. JEUSの設定ファイル

本付録では、domain.xmlの内容と項目について説明します。

A.1. 概要

OpenFrame GWをJEUSと接続するために、以下のディレクトリに格納されている**domain.xml**にサーバーとリソースの情報を設定します。

```
${JEUS_HOME}/domains/<domain_name>/config/
```

A.2. サーバーの追加

OpenFrame GWがインストールされるサーバーの情報をdomain.xmlの<server>タグに設定します。

```
<server>
  <lifecycle-invocation>
    <class-name>com.tmax.ofgw.Main</class-name>
    <invocation>
      <invocation-method>
        <method-name>init</method-name>
      </invocation-method>
      <invocation-type>READY</invocation-type>
    </invocation>
  </lifecycle-invocation>
  <lifecycle-invocation>
    <class-name>com.tmax.ofgw.Main</class-name>
    <invocation>
      <invocation-method>
        <method-name>shutdown</method-name>
      </invocation-method>
      <invocation-type>BEFORE_UNDEPLOY</invocation-type>
    </invocation>
  </lifecycle-invocation>

  <data-sources>
    <data-source>ds1</data-source>
  </data-sources>
</server>
```

以下は、各項目についての説明です。

項目	説明
<lifecycle-invocation>	ドメイン内のすべてのサーバーに各種ライフサイクル・イベントを指定することができます。
<invocation-method>	呼び出しに使用されるメソッドを指定します。
<invocation-type>	メソッドが呼び出される時点を指定します。 – READY : サーバーの起動後、登録されたアプリケーションがデプロイされ、サービスが準備された時点です。 – BEFORE_UNDEPLOY : サーバーの起動後、登録されたアプリケーションがデプロイされる前の時点です。
<data-sources>	サーバーで使用可能なデータソースを指定します。

A.3. リソースの追加

domain.xmlの<resources>タグにOpenFrame GWが使用するリソース情報を設定します。

```
</servers>
...
<resources>
<data-source>
  <database>
    <data-source-id>ds1</data-source-id>
    <export-name>ds1</export-name>
    <data-source-class-name>com.tmax.tibero.jdbc.ext.TbConnectionPoolDataSource
</data-source-class-name>
    <data-source-type>ConnectionPoolDataSource</data-source-type>
    <vendor>tibero</vendor>
    <server-name>192.168.33.225</server-name>
    <port-number>7629</port-number>
    <database-name>TVSAM</database-name>
    <user>tibero</user>
    <password>tmax</password>
    <login-timeout>0</login-timeout>
    <auto-commit>DRIVER</auto-commit>
    <stmt-query-timeout>0</stmt-query-timeout>
    <pool-destroy-timeout>10000</pool-destroy-timeout>
    <support-xa-emulation>false</support-xa-emulation>
    <connection-pool>
      <pooling>
        <min>2</min>
        <max>30</max>
        <step>1</step>
```



```

        <period>3600000</period>
    </pooling>
    <wait-free-connection>
        <enable-wait>false</enable-wait>
        <wait-time>10000</wait-time>
    </wait-free-connection>
    <max-use-count>0</max-use-count>
    <dba-timeout>-1</dba-timeout>
    <stmt-caching-size>-1</stmt-caching-size>
    <stmt-fetch-size>-1</stmt-fetch-size>
    <connection-trace>
        <enabled>false</enabled>
        <get-connection-trace>true</get-connection-trace>
        <auto-commit-trace>false</auto-commit-trace>
    </connection-trace>
    <use-sql-trace>false</use-sql-trace>
    <keep-connection-handle-open>false</keep-connection-handle-open>
</connection-pool>
</database>
</data-source>
</resources>
...
</domain>

```

以下は、各項目についての説明です。

項目	説明
<data-source-id>	データソースのIDを指定します。<server>に指定した<data-source>と同じく指定します。
<export-name>	データソースのJNDI名を指定します。未設定時には、データソースのIDをJNDI名として使用します。OpenFrame GW設定ファイル(ofgw.properties)のdatasource.nameと同じ名前を指定します。
<data-source-class-name>	JDBCドライバーのデータソース・クラス名(パッケージ名を含む)を指定します。
<data-source-type>	データソースのタイプを指定します。
<vendor>	JDBCドライバーのベンダー名を指定します。
<server-name>	データベースが実行されるホスト名またはIPアドレスを指定します。
<port-number>	データベース・リスナーのポート番号を指定します。
<database-name>	データベース名を指定します。
<user>	データベースのユーザーIDを指定します。
<password>	データベースのユーザー・パスワードを指定します。

参考

設定の詳細については、『JEUS XMLリファレンスガイド』を参照してください。

付録 B. Web通信

本付録では、OpenFrame GWでAAD(AIM Application Director)をサポートするためのOpenFrame GW RMIサーバーの構造と特徴について説明します。

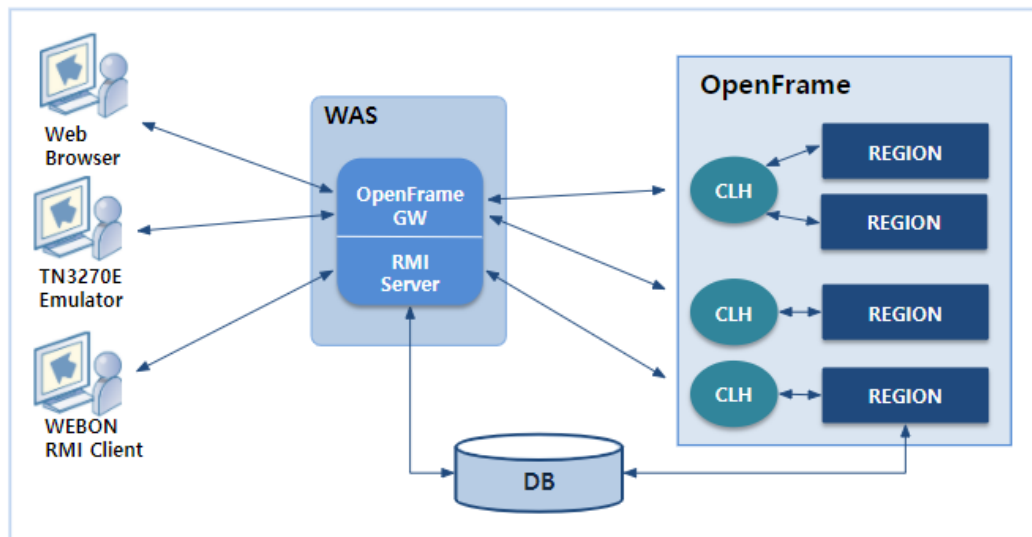
B.1. 概要

OpenFrame GW RMIサーバーは、InterstageのAADモジュールを代替するための機能です。Java RMI通信を使用して既存のADDの代わりにRMIサーバーを介してデータをやり取りします。RMIサーバーは、受信したデータをAIMのWEBAIMサービスに転送します。

B.2. 構造

OpenFrame GWは、WASのWebサーバーとして実装されますが、OpenFrame GWのRMIサーバーは、OpenFrame GWのように別途の製品ではなく、GWに含まれている機能です。

OpenFrame GWは、以前と同様にTN3270EエミュレーターとWebブラウザーからデータを受信してOpenFrameに転送し、RMIサーバーは、WEBON RMIクライアントからデータを受信してOpenFrame AIMのWEBAIMサービスに転送します。



[図 B.1] OpenFrame GW RMIサーバーの構成図

- WAS(Web Application Server)
OpenFrame GWは、WAS(JEUS)上で動作します。

- DB

現在接続されている端末、VTAM、オンライン・リージョンなどの情報に加え、RMIに受信されたクライアントの情報も保存します。

- CLH

Tmaxのクライアント・ハンドラーです。クライアントとサーバの間を中継し、サービスを提供する業務処理サーバーにサービスを要求し、サーバーへの接続および管理を行います。tpcallなどの関数によって受信されるクライアントの要求を該当するサーバーに転送します。

- REGION

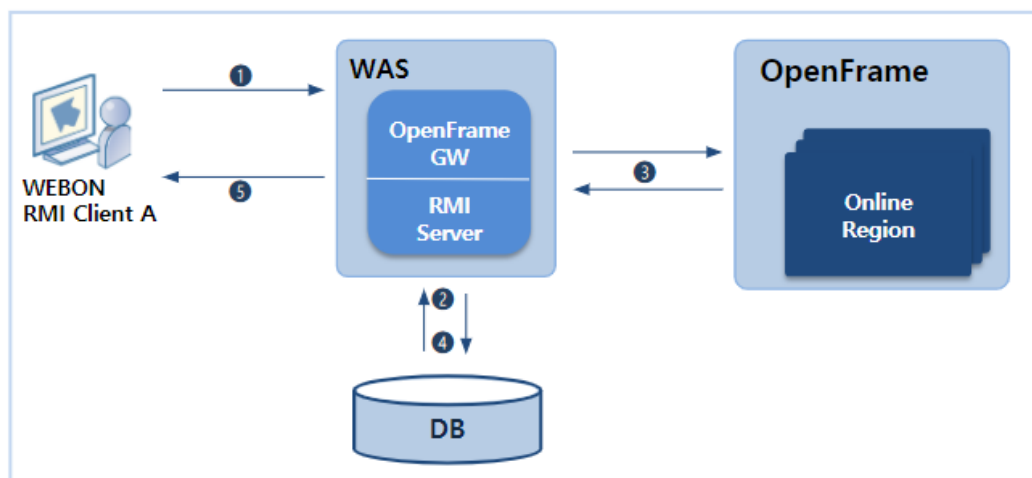
アプリケーション・プログラムを実行します。CLHを介してOpenFrame GWと通信します。

- WEBON RMI Client

WEBONは、Interstageに組み込まれているEJBで作成されたモジュールです。当該モジュールのRMIクライアントがOpenFrame GW RMIサーバーと通信します。

B.3. 動作プロセス

以下の図は、WEBON RMIクライアントからRMIサーバーにログオンし、オンライン・リージョンのWEBAIMサービスとトランザクションを実行するプロセスを示しています。(現在は、オンライン・リージョンとしてAIMのみサポートしています。)



[図 B.2] OpenFrame GW RMIサーバーの動作プロセス

1. WEBONモジュールがビジネス・ロジックの実行中に既存のAADと通信していたロジックが現れると、RMIサーバーにデータを転送します。
2. RMIサーバーは、当該接続を許可した後、DBにアクセスしてRMI Client Aの整合性をチェックしてからログオンを許可します。
3. RMIサーバーは、ofgw.propertiesに設定されているsmqn名とprocedure名を使用して受信したデータとトランザクションをオンライン・リージョンに転送し、実行結果を受け取ります。

4. OpenFrameのオンライン・リージョンとのトランザクションが完了したら、RMIサーバーはDBにアクセスしてRMI Client Aへのログイン情報を削除します。
5. OpenFrameのオンライン・リージョンから受信したデータをRMI Client Aに転送した後、RMI Client Aとの接続を終了します。

付録 C. OFGWメッセージ・コード

本付録では、OpenFrame GWから端末に出力されるメッセージ・コードについて説明します。

C.1. 概要

OpenFrame GWでは、端末に特定のイベント関連のメッセージを送信する場合にメッセージ・コードを使用します。

メッセージ・コードの形式は以下のとおりです。

```
MSG : OFGW{Message_Code} - {Message}
```

C.2. メッセージ・コード

以下は、OpenFrame GWから端末に送信されるメッセージ・コードについての説明です。

OFGW3000 - SERVER DOWN (3000)

説明	端末がログオンしたリージョンがダウンした場合に送信されます。
対応方法	OpenFrame GWと接続されているリージョンの状態を確認します。
参考	

OFGW3001 - {REGION_NAME} IS NOT FOUND. PLEASE RETRY. (3001)

説明	端末がログオンしようとするリージョンが見つからなかった場合に送信されます。
対応方法	OpenFrame GWと接続されているリージョンの状態と名前を確認します。 また、OpenFrame GWが接続されているDBからリージョン情報を正しく取得しているかどうかを確認します。
参考	

OFGW3002 - {LU_NAME} IS NOT FOUND. TRY ANOTHER LU. (3002)

説明	OpenFrame/OSC SD(System Definition)に定義された端末リソースのうち、接続した端末に割り当てられたLUに対応されるリソースが存在せず、AUTOINSTALLも設定されていない場合に送信されます。
対応方法	OpenFrame GWが接続されているDBから端末SDを正しく取得しているかどうかを確認します。また、OpenFrame/OSC SDの情報を確認します。
参考	

OFGW3003 - LOGON FAIL! PLEASE CHECK YOUR LU {LU_NAME}. (3003)

説明	OpenFrame/OSC SD(System Definition)に定義された端末リソースのうち、接続した端末に割り当てられたLUに対応されるリソースが存在せず、AUTOINSTALLも設定されていない場合に送信されます。
対応方法	OpenFrame GWが接続されているDBから端末SDを正しく取得しているかどうかを確認します。また、OpenFrame/OSC SDの情報を確認します。
参考	

OFGW3004 - THIS TERMINAL STATUS IS NOT IN SERVICE.(3004)

説明	端末に割り当てられたLUに対応されるOpenFrame/OSC SD(System Definition)の端末リソースがINSERVICEに設定されている場合に送信されます。
対応方法	OpenFrame/OSC SDの情報を確認し、SERVICEに設定してください。
参考	

OFGW3005 - THIS TERMINAL IS NO TTI.[{LU_NAME}] (3005)

説明	端末に割り当てられたLUに対応されるOpenFrame/OSC SD(System Definition)の端末リソースにTTIが設定されていない場合に送信されます。
対応方法	OpenFrame/OSC SDの情報を確認し、TTIを設定してください。
参考	

OFGW3006 - INVALID TRANSACTION. PLEASE RETRY.[{LU_NAME}] (3006)

説明	端末が入力したトランザクションを実行できない場合に送信されます。
対応方法	OpenFrame/OSC、OpenFrame/OSI、OpenFrame/AIMのトランザクション関連のシステム定義情報を確認します。また、OpenFrame GWが接続されているDBから各エンジンのトランザクション関連情報を正しく取得しているかどうかを確認します。
参考	

OFGW3007 - TMAX CHANNEL IS FULL. (3007)

説明	OpenFrame GWから端末にTmaxチャンネルを割り当てることができない場合に送信されます。
対応方法	ofgw.propertiesに定義されているTmaxチャンネルの設定情報のtmax.node.[NODENAME].maxの値を増やすか、チャンネルが割り当てられるまで再実行します。
参考	

OFGW3008 - TERMINAL RELEASED. (3008)

説明	端末がリージョンからログオフされたときに送信されます。
対応方法	
参考	ログオフされた場合に送信される情報メッセージです。

OFGW3009 - TMAX CHANNEL IS CLOSED.(3009)

説明	OpenFrame GWから端末にTmaxチャンネルの割り当てに失敗した場合に送信されます。
対応方法	OpenFrame GWに接続されているTmaxの状態を確認します。
参考	

**OFGW3010 - {REGION_NAME} IS NOT FOUND IN VTAM PROPERTY.
PLEASE RETRY.(3010)**

説明	端末がログオンしようとするリージョン名がvtam.propertiesに定義されていない場合に送信されます。
対応方法	vtam.propertiesのregionListに定義されているリージョン名を確認してください。
参考	

OFGW3011 - TERMINAL {LU_NAME} IS ONLY FOR AUTO INSTALL.(3011)

説明	端末に割り当てられたLUに対応されるOpenFrame/OSC SD(System Definition)の端末リソースがAUTOINSTALL ONLYに設定されている場合に送信されます。
対応方法	OpenFrame GWと接続されているOpenFrame/OSCのSD情報を確認します。
参考	